

Chapter 6* Geometry Practice

• 核心

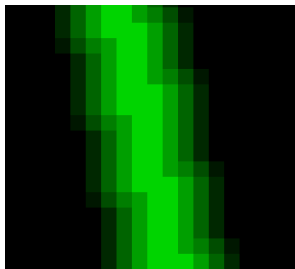
- **de Casteljau's algorithm**: 参考Chapter 6 Geometry中的算法解释即可
- **Bezier曲线反走样**

- **方案一**

- **内容**: 将Bezier曲线上的点的颜色依距离分配到与其相邻的四个点上
- **问题**: 仍会产生颜色的突变, 出现锯齿

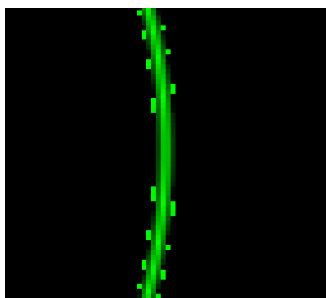
- **方案二**

- **内容**: 考虑一个以当前点为圆心, 具有一定半径的圆, 对于每个像素用四点采样法判断它是否在圆内。对于颜色的计算引入两个因子, 一是关于距圆心距离的衰减因子, 一是关于四点采样法采样比例的衰减因子。圆的半径可以根据实际的效果进行调整 (也许选择 $0.5x + \text{eps}$ 的半径更有助于形成平滑过渡的图像)
- **问题**: 我将半径设置为4.2, 将得到的Bezier曲线放大后仍然观测到了以下如图所示的情况。这可能是因为存在某些像素, 他们距离Bezier曲线上的点较远, 但是实际上他们距离这些点的连线并不远, 因此产生方案三

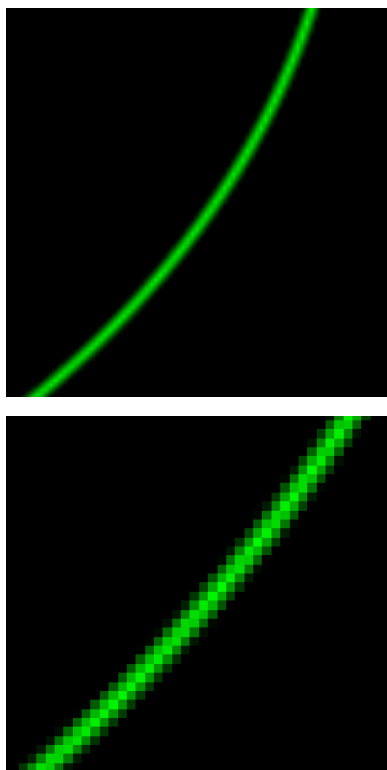


- **方案三**

- **内容**: 考虑将相邻两个Bezier曲线上的点的连线视作Bezier曲线在这点的切线, 将切线沿其垂直方向平移一定距离 d 后, 对两切线围成的封闭空间 (沿切线方向适当取长一些以确保每一段的连续性 (通过实际操作发现没必要)) 进行四点采样法, 然后再加上关于距切线距离 dis 的衰减因子 (与方案二类似)
- **问题**: 在进行简单实现后发现Bezier曲线边缘产生不应出现的G=255颗粒, 在将颗粒去掉后应得到较为光滑的Bezier曲线



- **解决：**关于距切线距离的衰减因子应当取为 $\min(1 - dis/d, 0)$ ，否则负数会导致意料之外的错误。最终效果图细节如下图所示，可见边缘过渡更为平滑，略微缩小后锯齿状边缘几乎消失。完整的效果图参见build文件夹下的my_bezier_curve



• 关于坐标变化

- **y轴方向上的对称：**对于①帧缓冲区 (frame_buf) 的输入；②纹理 (texture) 的提取需要对y坐标进行对称变换。因为点坐标是从原点开始，而在完成视口变换 (viewport transformation) 后帧缓冲区坐标和纹理坐标均从视口左上角从左往右从上往下进行编号
- **x,y交换：**这一般针对库中函数的接口定义，比如对于坐标(x,y)，他表示横坐标为x，纵坐标为y。但在x,y非负的情况下，它也可以表示第y行第x列。因此当接口要求传入的参数依次是行与列的话对于x,y坐标的交换是显而易见的

• 其它

• CV

- **cv::Mat**：一般用于存储将要在屏幕上显示的图像的每个像素点的颜色，原型为 `cv::Mat(rows, cols, type, cv::Scalar)`。 `cv::Scalar` 为矩阵初始化值，下面以 CV_8UC3 为例解释 `type` 的作用：①8代表8bit，即颜色深度为8bit；②U代表数据类型，即unsigned int无符号整型；③C3代表存储图片的通道数，即Channel 3，对应RGB三通道表示法
- **cv::cvtColor**：用于颜色空间的转换。颜色空间参考 `cv::Mat` 中对于C3的解释，另外这里的转换也涉及对通道顺序的转换。函数原型为 `cv::cvtColor(InputArray, OutputArray, code)`，其中 `code` 为转换码，如 `BGR2RGB` 的意思为"BGR to RGB"，即将BGR通道转换为RGB通道
 - **注1：**从实践结果来看，`cv::cvtColor(InputArray, OutputArray, RGB2BGR)` 的作用并不是直接改变RGB三个通道所对应的比特，即在执行 `cvtColor` 之后所有加入

`cv::Mat` 中的颜色都以RGB的顺序排列。实际上他只是交换了R通道和B通道下的数字，因此这个指令在初始化时对`cv::Mat`使用并无作用，而应在 `cv::imshow` 之前使用

- **注2：**再次实践发现对于不同类型通道的转换则可以在初始化时实现，推测若想改变颜色空间通道顺序则必须在 `cv::imshow` 之前调用 `cv::cvtColor`，因为这本质上是调换通道内数值的顺序；而若想改变颜色空间类型则可以在初始化时改变。于是可以得出结论在OpenCV中，任何情况下三通道颜色的存储都是以BGR顺序存储的，

`RGB2BGR` 只能改变存储数值的位置，并不能从根本上将其顺序变为RGB

- **`cv::namedWindow`**：用于显示窗口一些属性的设定，原型为 `cv::namedWindow(winname, flags)`，`winname` 为显示窗口的名称，而 `flags` 为窗口拉伸属性，如 `cv::WINDOW_AUTOSIZE` 为自动调整大小，不可人为调整
- **`cv::setMouseCallback`**：用于设置对鼠标输入做出的回应，这里不做细致阐述